UNIVERSITY OF ATHENS
DEPARTMENT OF INFORMATICS AND TELECOMMUNICATIONS

Deep Learning for NLP

Student name: *Kleanthis Vasileiadis*
sdi: 1115202200017

Course: *Artificial Intelligence II (M138, M226, M262, M325)*
Semester: *spring Semester 2025*

Contents

1 Abstract **3**

2 Data processing and analysis **3**
2.1 Pre-processing . . . 3
2.2 Analysis . . . 4
2.2.1 **Analysis Before Text Pre-processing** . . . 5
2.2.2 **Analysis After Text Pre-processing** . . . 6
2.3 Data partitioning for train, test and validation . . . 9
2.4 Vectorization . . . 10

3 Algorithms and Experiments **10**
3.1 Experiments . . . 10
3.1.1 Experiment 1 - Baseline model . . . 11
3.1.2 Experiment 2 - Adding text pre-processing . . . 12
3.1.3 Experiment 3 - Text preprocessing that reduced accyracy . . . 13
3.1.4 Experiment 4 - Incorrectly tuned hyperparameters(underfitting) . 14
3.1.5 Experiment 5 - Incorectly tuned hyperparameters(overfitting) . . 15
3.1.6 Experiment 6 - Training the model on both the training and validation datasets . . . 17
3.1.7 Experiment 7 - Quick briefing of other techniques . . . 17
3.1.8 Table of trials . . . 18
3.2 Hyper-parameter tuning . . . 18
3.2.1 Key Hyper-parameters . . . 18
3.2.2 Hyper-parameters tuning results . . . 19
3.2.3 Prevention of overfitting and underfitting . . . 20
3.3 Optimization techniques . . . 20
3.4 Evaluation . . . 20
3.4.1 ROC curve . . . 21
3.4.2 Learning Curve . . . 22
3.4.3 Clasiffication report . . . 23
3.4.4 Confusion matrix . . . 24

4 Results and Overall Analysis **25**
4.1 Results Analysis 25
4.1.1 Best trial 26

5 Bibliography **26**

1. Abstract

This project is aimed to develop a sentiment classifier using machine learning and AI algorithms in Python on a given English-language Twitter dataset. The dataset consists of three main components: an ID, which uniquely identifies each tweet, the text containing the actual content of the tweet and the label which represents the sentiment, where 1 indicates a positive sentiment and 0 indicates a negative one. The process of classifying the tweets involves sorting them into these two distinct categories based on their sentiment. To achieve this, various natural language processing (NLP) techniques will be employed, including text preprocessing, feature extraction methods and the implementation of a classification model. The ultimate goal of this project is to create an efficient and accurate classifier capable of distinguishing between positive and negative sentiments in real-world Twitter data.

2. Data processing and analysis

2.1. Pre-processing

Pre-processing is a crucial step in machine learning and AI applications, as it ensures that the input data is clean, consistent, and suitable for training a sentiment classification model. In this project, several data cleaning and text pre-processing techniques were applied to the raw Twitter dataset to improve the quality of the textual data and enhance model performance.
The main steps involved in the pre-processing phase include:

- **Replacing emojis**: Converts emojis into the word "emoji".
- **Replacing emoticons**: Converts emoticons into their corresponding text representations.
- **Replacing HTML entities**: Converts HTML entities into readable text or their corresponding special character.
- **Replacing mentions**: Replaces mentions with the word "mention".
- **Replacing links**: Replaces URLs starting with "http","https" and "www" with the word "link".
- **Replacing money amounts**: Replaces money values with the word "money".
- **Replacing dates and months**: Converts standard date format(day/month/year) into the word "date" and month names into the word "month".
- **Replacing emails**: Replaces email addresses with the word "email".
- **Replacing phone numbers**: Replaces phone numbers of various formats(e.g., with country codes, dashes, or spaces) with the word "phone".
- **Replacing credit card numbers**: Replaces credit card numbers(13 to 16 digits, with optional spaces or dashes) with the word "creditcard".

- **Replacing IP addresses**: Replaces IP addresses(in the format xxx.xxx.xxx.xxx) with the word "ip".
- **Replacing time expressions**: Replaces time formats (e.g., "12:30 PM") with the word "time".
- **Replacing numbers**: Replaces the remaining numbers with the word "number".
- **Replacing names**: Replaces names with the word "name".
- **Replacing places**: Replaces named places (e.g.,countries, cities) with the word "place".
- **Replacing terrains**: Replacing geographic features (e.g., mountains, rivers) with "terrain".
- **Removing special characters**: Keeps only alphabetic characters and spaces.
- **Restricting repeated characters**: Restricting repeated consecutive characters to a maximum of two (e.g.,"looooooove"->"loove").
- **Normalizing text to lowercase**: Converts all text to lowercase for consistency.
- **Handling negations**: Detects and processes negations properly.
- **Tokenization**: Splits text into individual words or subwords, which is essential for model training

The process of replacing emoticons, HTML entities, names, places, and terrains was carried out using custom lists and dictionaries, which were carefully constructed by analyzing the training dataset. All other replacements were performed using regular expressions (regex) with the "re" library. The handling of negation was done manually by identifying specific negation words extracted from the training dataset and appending a "not_" prefix to the subsequent word. Proper pre-processing not only standardizes the text but also helps reduce noise in the data, making overfitting less likely in the classification model.

2.2. Analysis

Before proceeding with model training, it is essential to analyze and visualize the dataset to gain insights into its structure and characteristics. This step involves exploring the raw text data, identifying patterns, and understanding the distribution of words and sentences. By performing this analysis both before and after preprocessing, we can evaluate the impact of text cleaning techniques and ensure that important information is preserved while noise is reduced. To achieve this, several exploratory data analysis (EDA) techniques are employed, including text visualizations, frequency distributions, and statistical summaries. These analyses help reveal key patterns in the dataset and guide further improvements in preprocessing and feature extraction.

2.2.1. Analysis Before Text Pre-processing. The first step is to explore the raw data and gain a better understanding of its composition. This involves printing the first few records and analyzing their structure, class distribution, and missing values. By reviewing these aspects, it can be decided what preprocessing steps are needed. Additionally, checking the class distribution helps to ensure that the dataset is balanced or identify any imbalances that may need to be addressed.

```
Training data
       ID                                                Text  Label
0  189385      @whoisralphie dude  I'm so bummed ur leaving!      0
1   58036  oh my god, a severed foot was foun in a wheely...      0
2  190139  I end up &quot;dog dialing&quot; sumtimes. Wha...      1
3   99313                          @_rachelx meeeee toooooo!      0
4  157825  I was hoping I could stay home and work today,...      0


Validation data
       ID                                                Text  Label
0  187062  @NatexTheGreat heyheyhey maybe i will  get the...      0
1  168997  @molliesmummy Work? Work was crap. I missed yo...      0
2  194461  Want: Trip to Boston next month. Need: Addit'l...      1
3  165442                         first day starts tomorrow!      1
4   34853  @goodforyoursoul 8 course fish in Little Saigo...      1


Testing data
       ID                                                Text
0  102388  The books ARRIVED!!! Those who want a copy - w...
1  153802  @Spunk_Ransom_09 alright yea its cute. I guess...
2   61831        On the way to zen buffet! no crabs I think
3   18317             Good Tuesday Morning All (5:20am)....
4   50061                                        Bed timeee
```

Unprocessed data

As observed, many of the pre-processing steps discussed in the previous section are reflected in the data. In the following, the dataset statistics will be presented along with some plots to further analyze the data.

```
Train data info:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 148388 entries, 0 to 148387
Data columns (total 3 columns):
 #   Column  Non-Null Count   Dtype
---  ------  --------------   -----
 0   ID      148388 non-null  int64
 1   Text    148388 non-null  object
 2   Label   148388 non-null  int64
dtypes: int64(2), object(1)
memory usage: 3.4+ MB
None

Train data missing values:
ID       0
Text     0
Label    0
dtype: int64

Train data frequency:
Label
1    74196
0    74192
Name: count, dtype: int64

Train data statistics:
Label
1    0.500013
0    0.499987
Name: proportion, dtype: float64
```

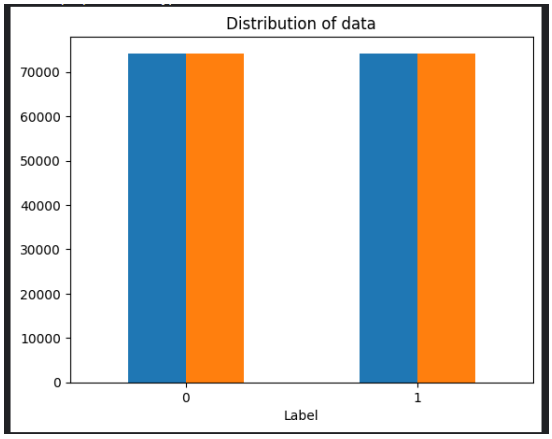
(a) Training Data Statistics

```
Valuation data info:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 42396 entries, 0 to 42395
Data columns (total 3 columns):
 #   Column  Non-Null Count  Dtype
---  ------  --------------  -----
 0   ID      42396 non-null  int64
 1   Text    42396 non-null  object
 2   Label   42396 non-null  int64
dtypes: int64(2), object(1)
memory usage: 993.8+ KB
None

Valuation data missing values:
ID       0
Text     0
Label    0
dtype: int64

Valuation data frequency:
Label
1    21199
0    21197
Name: count, dtype: int64

Valuation data statistics:
Label
1    0.500024
0    0.499976
Name: proportion, dtype: float64
```

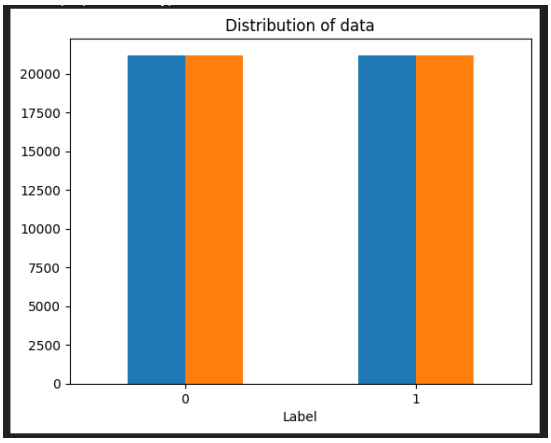
(b) Validation Data Statistics

```
Test data info:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 21199 entries, 0 to 21198
Data columns (total 2 columns):
 #   Column  Non-Null Count  Dtype
---  ------  --------------  -----
 0   ID      21199 non-null  int64
 1   Text    21199 non-null  object
dtypes: int64(1), object(1)
memory usage: 331.4+ KB
None

Test data missing values:
ID      0
Text    0
dtype: int64
```

(c) Test Data Statistics



(d) Training Data Distribution



(e) Validation Data Distribution

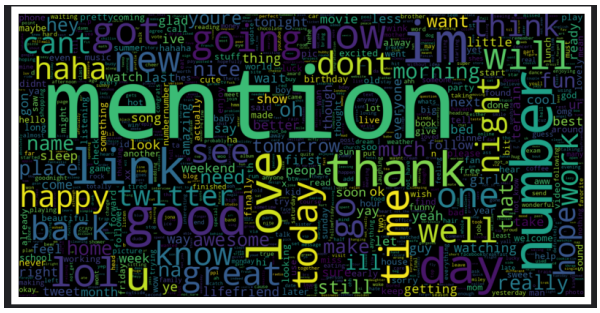
Upon analyzing each dataset's statistics, it is evident that the classes are balanced, with a total of 211,983 tweets. Additionally, there are no missing values present in any of the datasets.

2.2.2. Analysis After Text Pre-processing. After applying the pre-processing techniques outlined in the previous section, it is essential to examine the transformed dataset. This involves inspecting the cleaned text to ensure that unnecessary noise has been removed while preserving meaningful information. Additionally, visualizing word distributions and token frequencies provides insights into how the text has been processed. By comparing these findings with the initial dataset, the impact of the pre-processing steps can be assessed, ensuring that the text is well-prepared for subsequent feature extraction and model training. Next, word clouds and frequency plots will be presented for each label in the training and validation datasets to provide a visual representation of the most common words after pre-processing.

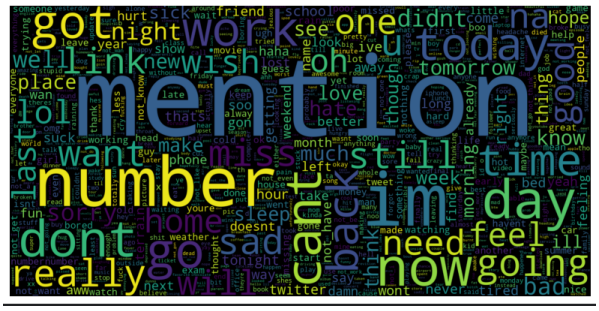
```
Cleaned training data
0                 mention dude im so bummed ur leaving
1    oh my god a severed foot was foun in a wheely ...
2    i end up dog dialing sumtimes whats dog dialin...
3                                       mention mee too
4    i was hoping i could stay home and work today ...
5    says plurk karma finally reached the s still n...
6    good to hear it mention this is a bit more but...
7    mention in that case im gon na start tweeting ...
8                 mention its alright love how are you
9    mention haha that has to be the best analogy e...
Name: Text, dtype: object

Cleaned validation data
0    mention heyheyhey maybe i will get them if i c...
1    mention work work was crap i missed your conde...
2    want trip to place next month need additl moti...
3                             first day starts tomorrow
4    mention number course fish in little saigon an...
5    no not_milk for breakfast grr but looks like a...
6    mention im signed up to get news updates on it...
7    mention mention im working tonight but i might...
8             been to work this morning jolly good fun
9    mention i have lived here for number years thi...
Name: Text, dtype: object

Cleaned test data
0    the books arrived those who want a copy write ...
1    mention alright yea its cute i guess this prov...
2        on the way to zen buffet no not_crabs i think
3                    good tuesday morning all numberam
4                                             bed timee
5    german government got squezzed out by bancrupt...
6    link mention mention nice seeing you guys agai...
7    mention i can smell it too enjoy and keep us u...
8                  good morning everyone happy fridayy
9    mention teehee well done on your bronze medall...
Name: Text, dtype: object
```

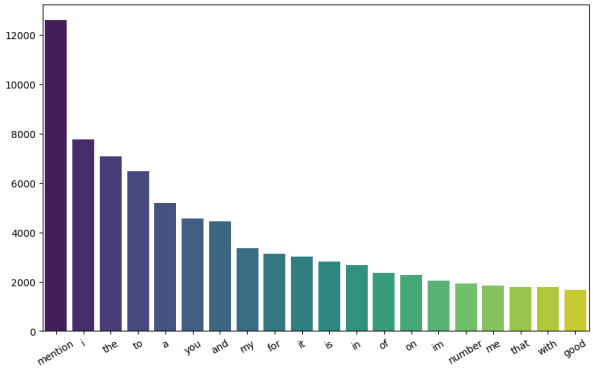
Processed data

Upon examining the data after text pre-processing, it becomes evident that a significant amount of noise present in the raw text has been effectively removed or replaced. This improvement enhances the clarity and consistency of the dataset, ensuring that only relevant textual information is retained for further analysis and model training.

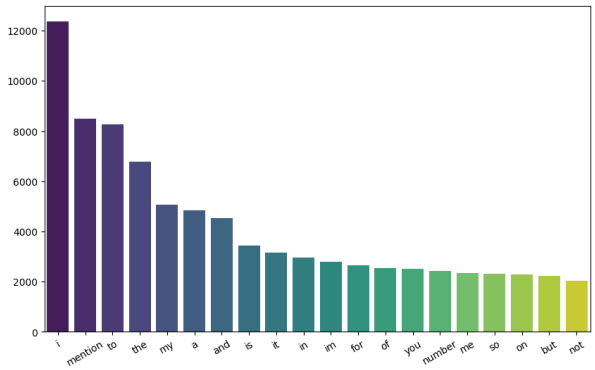


(a) Positive Words On Training Dataset

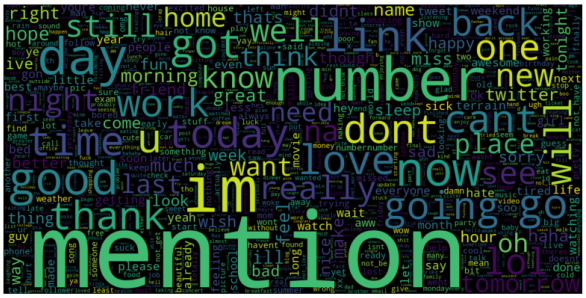

mention
number
im
day
now
going
need
feel
dont
really
go
sad
cant
back
miss
want
work
got
one
didnt
today
tomorrow
still
me
know
good
bad
lol
place
friend
night
see
hope
people
twitter
damn
wont
never
tired
aww
watch
sick
weekend
hate
thing
morning
already
right
week
bed
year
fun
sleep
home
sorry
hour
doesnt
not
have
make
phone
left
okay
tweet
house
made
watching
weather
shit
sick
sleep
tonight
way
not know
not
book
book
play
pretty
coming
maybe
call
movie
less
want
little
ready
hurt
sick
friend
school
hope
people
thing
good
know
bad
place
lol
suck
working
head
cold
make
phone
month
left
okay
tweet
house
made
watching
weather
shit
sick
sleep
tonight
way
not know
not
book
book
play
pretty
coming
maybe
call
movie
less
want
little
ready
hurt
sick
friend
school
hope
people
twitter
damn
wont
never
tired
aww
watch
sick
weekend
hate
thing
morning
already
right
week
bed
year
fun
sleep
home
sorry
hour
doesnt
not
have
make
phone



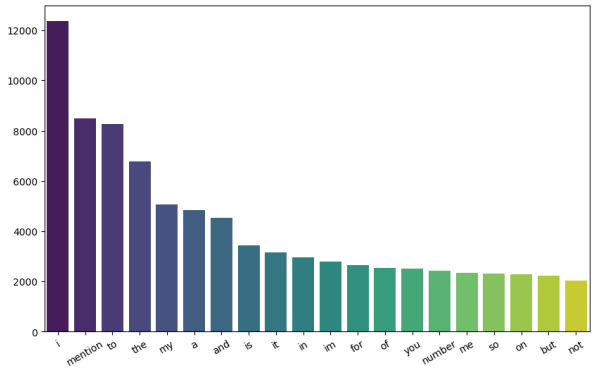
(a) Positive Words Frequency On Validation Dataset



(b) Negative Words Frequency On Validation Dataset



(a) Word Cloud On Testing Dataset



(b) Words Frequency On Testing Dataset

By examining the word clouds and frequency plots for each label across the training and validation datasets, it is observed that while the top 20 most frequent words are largely similar across both sentiment classes, the word clouds reveal noticeable differences in the broader vocabulary used in positive and negative tweets. This indicates that although common words appear in both categories, their contextual usage and the presence of sentiment-specific terms contribute to the distinction between the two classes. Moreover, the consistency in word distributions across datasets suggests that the pre-processing steps were applied uniformly, ensuring data integrity. The retention of stopwords, which improves model accuracy, may explain the overlap in frequent words, yet the overall variation in less frequent words still provides valuable signals for sentiment classification. These findings confirm that the dataset is well-prepared for training, preserving the necessary information while eliminating noise.

2.3. Data partitioning for train, test and validation

The dataset was provided pre-partitioned into training, validation, and test sets. The distribution follows a standard and well-balanced ratio: approximately 70 percent of the data (149K tweets) is used for training, 20 percent (42K tweets) for validation, and 10 percent (22K tweets) for testing.

This split ensures a well-balanced approach, allowing the model to learn effectively from a large portion of the data while keeping sufficient samples for hyperparameter tuning and evaluation. The validation set helps in preventing overfitting by guiding model adjustments, while the independent test set provides a final assessment of per-

formance. These ratios are widely used in deep learning tasks, particularly for large NLP datasets, ensuring both reliable training and robust evaluation.

2.4. Vectorization

To transform the raw textual data into a numerical format that can be processed by machine learning models, TF-IDF (Term Frequency-Inverse Document Frequency) was employed. TF-IDF is a statistical measure used to evaluate the importance of a word in a document relative to the entire corpus. It works by calculating two components:

- **Term Frequency (TF)**: Measures how frequently a word appears in a document. It's essentially the raw count of a term divided by the total number of terms in the document.
- **Inverse Document Frequency (IDF)**: Measures the importance of the term within the whole corpus. If a word appears in many documents, its IDF value decreases, indicating it's less informative for distinguishing between documents.

The TF-IDF score is the product of these two components, giving higher weight to words that appear frequently in a specific document but are less common across the entire corpus. This ensures that terms that are more unique and contextually important are prioritized.

This method was chosen because it effectively captures the significance of words without being overly influenced by common, frequently occurring words (e.g., "the", "and"), which may not provide meaningful insights in sentiment analysis. By assigning higher weight to less frequent, contextually important terms, TF-IDF helps to emphasize the unique characteristics of the text relevant to sentiment classification.

Before applying TF-IDF, the text was preprocessed to ensure consistency and eliminate noise as described in the previous sections. The resulting numerical vectors, each representing a document in a high-dimensional space, are then used as input features for the machine learning model to predict sentiment.

This approach ensures that the model can efficiently learn from the significant features of the text while minimizing the influence of irrelevant or overly common words.

3. Algorithms and Experiments

3.1. Experiments

Building an effective classification model requires iterative experimentation to refine its performance. This section details the approach taken to optimize the model, starting with an initial baseline run followed by systematic improvements through various techniques. The goal was to identify methods that enhance generalization, improve accuracy, and mitigate overfitting.

Given the large number of experiments conducted throughout the process, it is not feasible to present every single one. Instead, this section highlights a selection of representative experiments that best illustrate the impact of different optimization strategies. Both successful and unsuccessful attempts are discussed, providing insights into how various techniques influenced model performance. Additionally, evaluation metrics such as learning curves, confusion matrices, and ROC curves are presented to visualize the impact of these optimizations.

3.1.1. Experiment 1 - Baseline model. The first experiment involved training a simple baseline model using logistic regression without any hyperparameter tuning or text preprocessing. The TF-IDF vectorization was applied with its default settings, meaning no specific adjustments were made to parameters such as maximum features or n-grams. Similarly, the logistic regression classifier was used with its default configuration. Due to the lack of hyperparameter tuning, the model exhibited signs of overfitting, which will be further analyzed in the section presenting overfitting.

This initial experiment serves as a reference point, providing insight into the dataset's learnability and the model's raw performance before applying any optimizations. By establishing this baseline, we can later compare the impact of different improvements such as text preprocessing, hyperparameter tuning, feature selection, and optimization techniques.

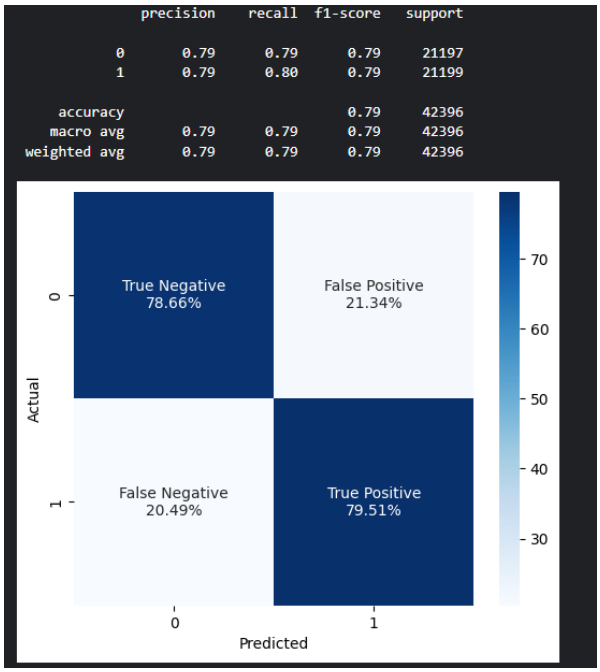
The baseline model achieved an accuracy of 79 percent on the validation dataset and 78,937 percent on the test dataset. While these results indicate that the model is already capturing some patterns in the data, there is still room for improvement.

Evaluation And Plots:

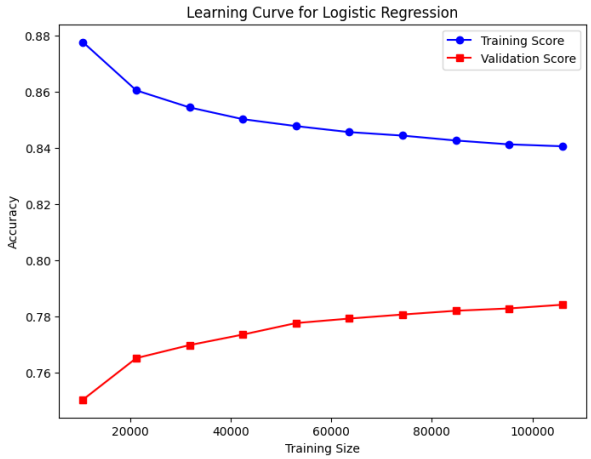
- **Classification Report and Confusion Matrix**: The classification report and confusion matrix demonstrate that the model achieves a reasonable baseline accuracy, along with good prediction rates for true positives, true negatives, false positives, and false negatives. However, there is still room for improvement to further enhance the model's performance.

- **Learning Curve**: The learning curve reveals that, due to the lack of regularization, the baseline model is overfitting. This is evident from the large gap between training and validation performance, suggesting that the model has memorized the training data rather than generalizing well to unseen data.

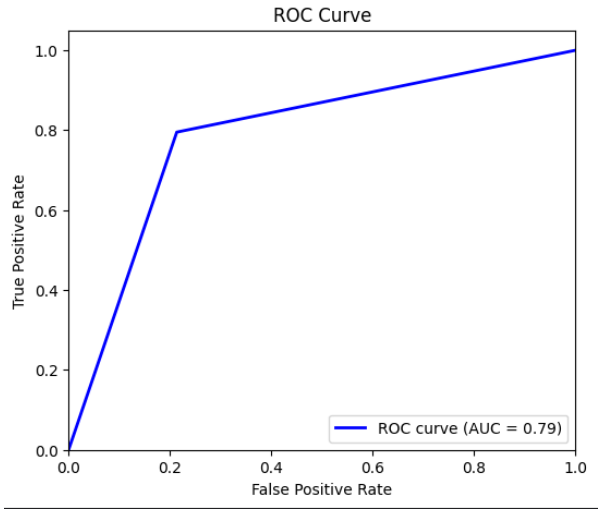
- **ROC Curve**: The ROC curve illustrates the model's ability to distinguish between the classes, showing the trade-off between the true positive rate (sensitivity) and the false positive rate (1-specificity). A steeper curve closer to the top-left corner indicates better performance. The baseline model shows a reasonable ability to discriminate between classes, but it can likely be improved further with optimization techniques.



(a) Clasification Report And Confusion Matrix



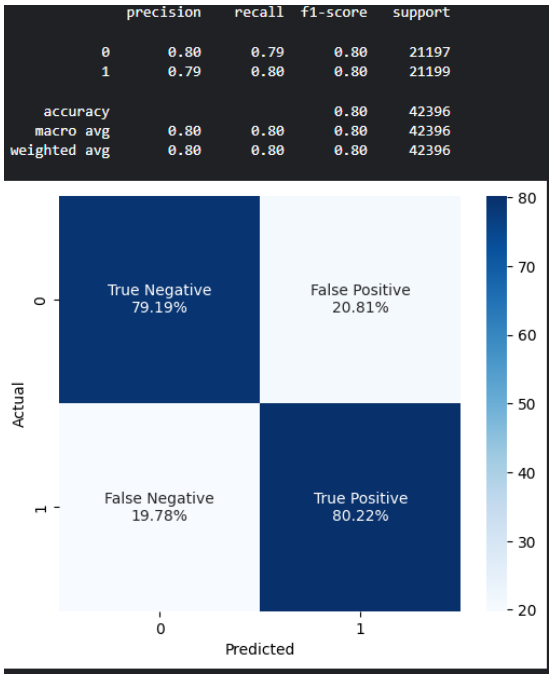
(b) Learning curve



(c) ROC curve

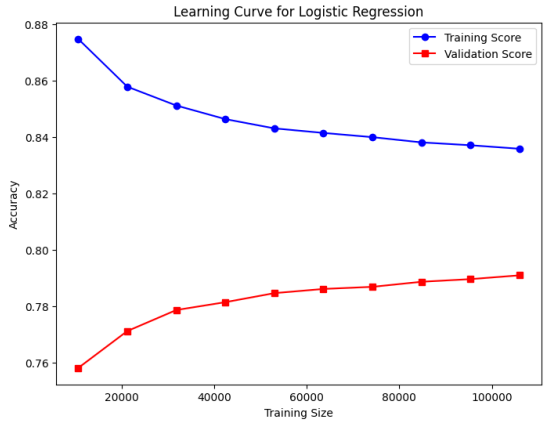
3.1.2. Experiment 2 - Adding text pre-processing. In Experiment 2, i incorporated text pre-processing techniques to enhance the model's performance by reducing noise and helping the model focus on the most relevant features of the data. This experiment builds upon the baseline model from Experiment 1, with the key difference being the inclusion of various text cleaning and transformation steps. These pre-processing steps, as outlined in Section 2, aim to standardize the text data, remove unnecessary information, and better prepare it for the machine learning model. This model achieved a slight improvement in accuracy and other statistics (such as F1 score, recall, etc.) compared to Experiment 1. However, it did not show any noticeable reduction in overfitting, as observed in the learning curve.

This model achieved an accuracy of 80 percent on the validation dataset and 78.947 percent on the test dataset. While there is a slight improvement in the metrics, there is still room for further optimization.

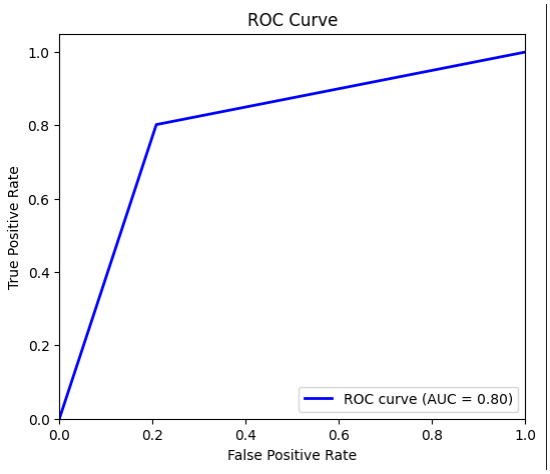


	precision	recall	f1-score	support
0	0.80	0.79	0.80	21197
1	0.79	0.80	0.80	21199
accuracy			0.80	42396
macro avg	0.80	0.80	0.80	42396
weighted avg	0.80	0.80	0.80	42396

(a) Clasification Report And Confusion Matrix

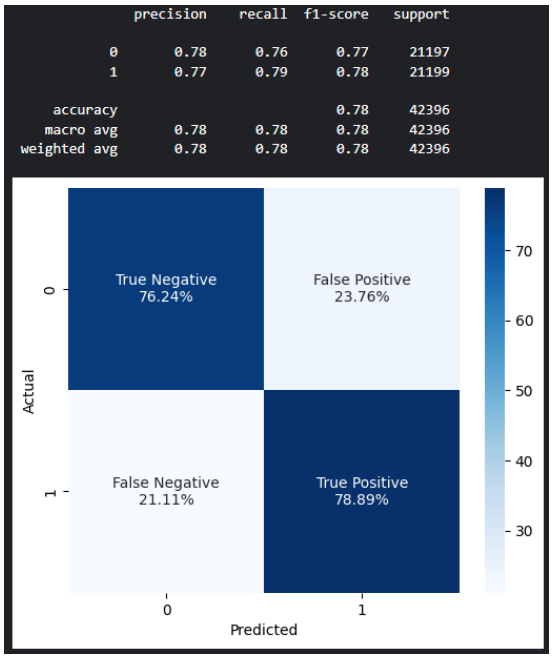


(b) Learning curve



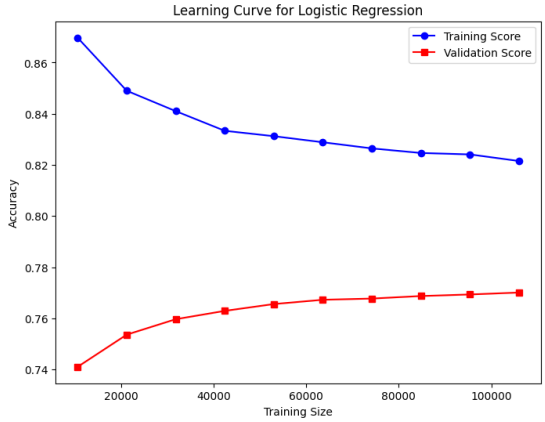
(c) ROC curve

3.1.3. Experiment 3 - Text preprocessing that reduced accyracy. In Experiment 3, additional text preprocessing steps were applied, including the removal of stopwords using the default stopwords list from TF-IDF and the application of stemming. However, these preprocessing steps led to a reduction in accuracy compared to previous experiments. The removal of stopwords, while helping to eliminate common but less informative words, removed some context that contributed to the model's performance. Similarly, stemming, which reduces words to their root form, also removed some important nuances from the data, negatively impacting the model's ability to make accurate predictions. In addition, while the accuracy and other metrics decreased, overfitting was still evident, making this approach ineffective overall.
This model achieved an accuracy of 78 percent on the validation dataset and 73.022 percent on the test dataset, making it the worst-performing model so far.

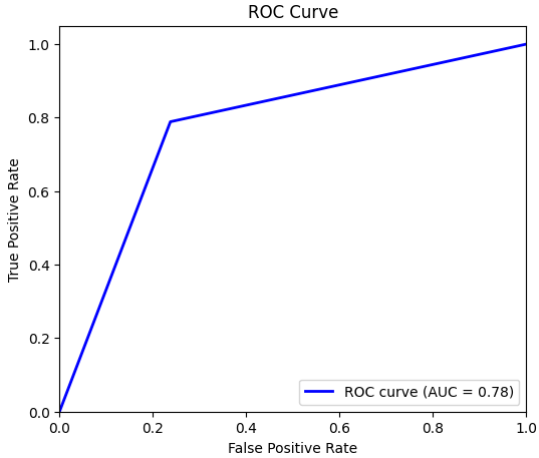


	precision	recall	f1-score	support
0	0.78	0.76	0.77	21197
1	0.77	0.79	0.78	21199
accuracy			0.78	42396
macro avg	0.78	0.78	0.78	42396
weighted avg	0.78	0.78	0.78	42396

(a) Clasification Report And Confusion Matrix



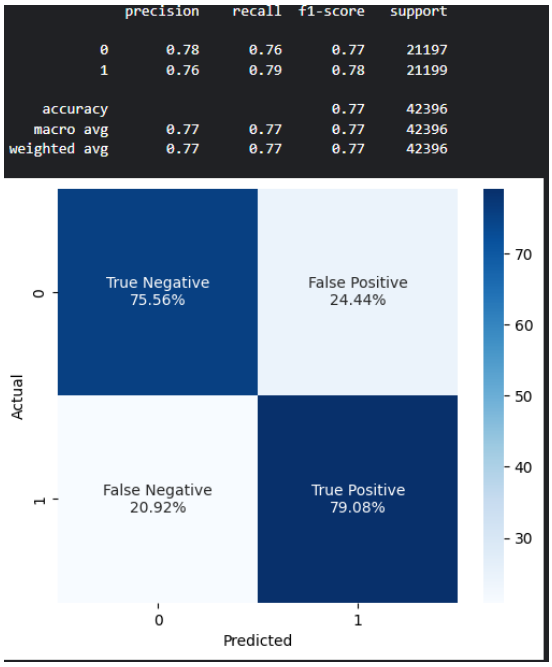
(b) Learning curve



(c) ROC curve

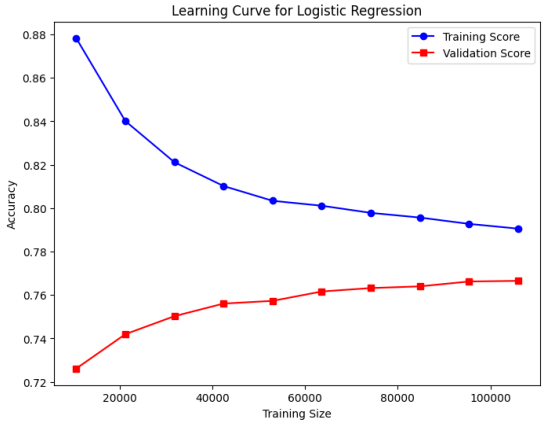
3.1.4. Experiment 4 - Incorrectly tuned hyperparameters(underfitting). In Experiment 4, hyperparameter tuning was deliberately misconfigured to analyze the impact of poorly chosen values on model performance, despite using the optimal text preprocessing steps. The TF-IDF vectorizer parameters were set with max_df=0.7, min_df=5, max_features=5000, and ngram_range=(1,3), which overly restricted the feature space, potentially discarding useful terms. Additionally, the logistic regression model was configured with C=6 , limiting its ability to learn complex patterns from the data. As expected, these suboptimal choices led to underfitting, reducing accuracy and other evaluation metrics. The learning curve further confirms this issue, showing that both training and validation errors remained high, indicating that the model failed to capture the dataset's complexity. Similarly, the confusion matrix highlights a drop in correct predictions, suggesting a weaker distinction between classes. The ROC curve also reflects this degradation, showing a lower area under the curve (AUC), which indicates a reduced ability to separate positive and negative classes effectively. This experiment

reinforces the importance of proper hyperparameter tuning, as overly restrictive configurations can significantly hinder predictive performance.
This model achieved an accuracy of 77 percent on the validation dataset and 72.508 percent on the test dataset, making it the worst-performing model so far.

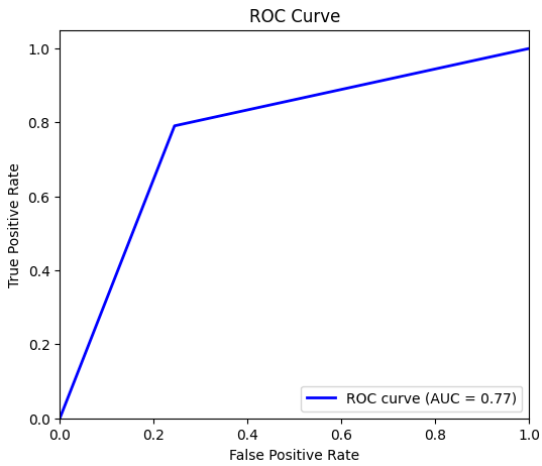


	precision	recall	f1-score	support
0	0.78	0.76	0.77	21197
1	0.76	0.79	0.78	21199
accuracy			0.77	42396
macro avg	0.77	0.77	0.77	42396
weighted avg	0.77	0.77	0.77	42396

(a) Clasification Report And Confusion Matrix



(b) Learning curve



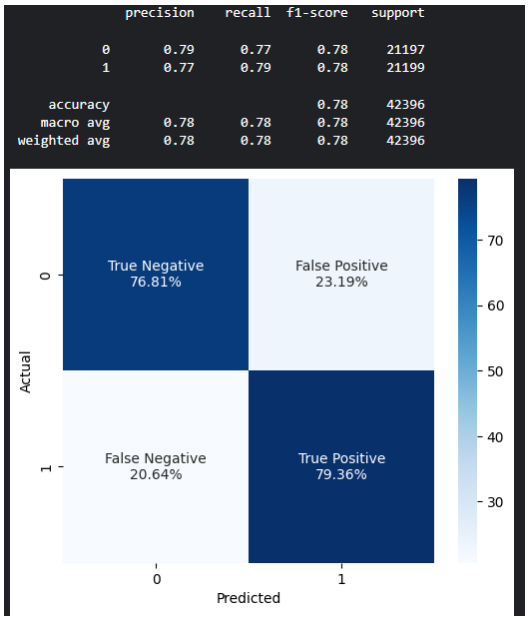
(c) ROC curve

3.1.5. Experiment 5 - Incorectly tuned hyperparameters(overfitting). In Experiment 5, hyperparameter tuning was again deliberately misconfigured, but this time in a way that led to overfitting. While using the optimal text preprocessing steps, the TF-IDF vectorizer parameters were adjusted to max_df=0.95, min_df=2, max_features=None meaning all features were used, and ngram_range=(1,3), significantly increasing the feature space and capturing more complex patterns, including potentially noisy or less relevant terms. Additionally, the logistic regression model was configured with C=1.5, reducing regularization and allowing the model to fit the training data too closely.

As a result, the model demonstrated a higher accuracy compared to previous experiments; however, the severe overfitting rendered it ineffective. The learning curve clearly illustrates this issue, showing a substantial gap between training and validation accuracy, indicating that the model memorized patterns from the training data rather than learning generalizable features. The confusion matrix, evaluated on the validation dataset, reveals a noticeable drop in correct predictions despite the high training performance, further confirming that the model struggles to generalize. Similarly, the ROC curve on the validation set shows a decline in AUC, highlighting a weaker distinction between classes when applied to new data.

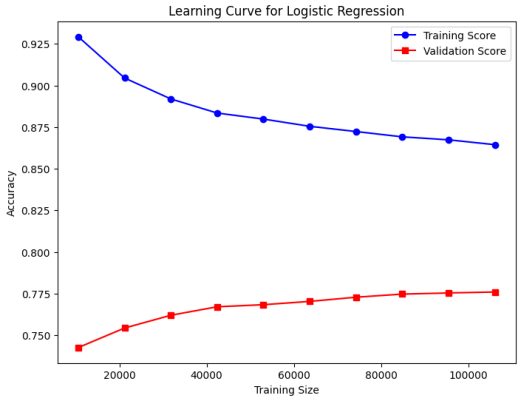
This experiment underscores that while increasing accuracy might seem beneficial, excessive model complexity and insufficient regularization can lead to overfitting, ultimately reducing the model's reliability for real-world predictions.

This model achieved an accuracy of 78 percent on the validation dataset and 80.834 percent on the test dataset, making it the highest-scoring model so far in terms of metrics. However, despite the improved accuracy, the presence of overfitting limits its effectiveness as a reliable classification model.

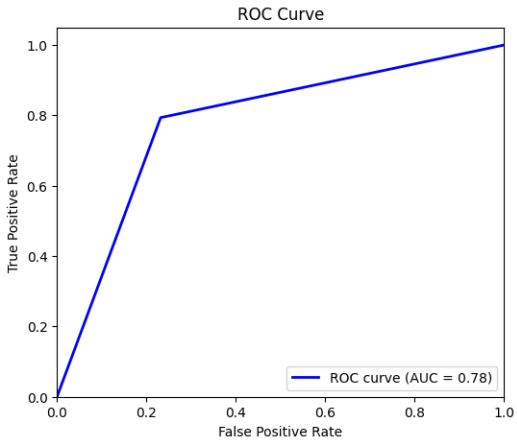


	precision	recall	f1-score	support
0	0.79	0.77	0.78	21197
1	0.77	0.79	0.78	21199
accuracy			0.78	42396
macro avg	0.78	0.78	0.78	42396
weighted avg	0.78	0.78	0.78	42396

(a) Clasification Report And Confusion Matrix



(b) Learning curve



(c) ROC curve

3.1.6. Experiment 6 - Training the model on both the training and validation datasets. In Experiment 6, the model was trained using both the training and validation datasets, increasing the total amount of available training data. The goal was to examine whether incorporating the validation set into training would lead to improved performance on the test set by providing the model with more data to learn from.

As expected, this approach resulted in the highest accuracy recorded across all experiments, as the model had access to a larger portion of the dataset during training, indicating that the additional training data helped the model generalize better.

However, since the validation set was used for training, there is no longer an independent validation dataset available for evaluation. As a result, it is not possible to generate meaningful plots such as learning curves, confusion matrices, or ROC curves, as these visualizations rely on separate validation data to assess model performance. Plotting metrics solely on the training data would not provide useful insights, as it would only reflect how well the model memorized the training set rather than its generalization ability.

This experiment highlights the benefits of leveraging more data for training. However, it also emphasizes the importance of maintaining a separate validation set for unbiased evaluation, as using the validation data for training means that a new method for assessing model performance is required.

This model achieved an accuracy of 81.47 percent on the test dataset, making it the highest-scoring model I managed to achieve in terms of metrics. However, despite the improved accuracy, the absence of a validation dataset—and consequently, the inability to generate evaluation metrics—limits its effectiveness as a reliable classification model. Without an independent validation set, it is difficult to assess the model's true generalization ability, making this approach less ideal despite the high test accuracy.

3.1.7. Experiment 7 - Quick briefing of other techniques. In Experiment 7, I briefly reviewed several additional text preprocessing techniques. While some of these techniques did show improvements in various evaluation metrics, they were not optimal, and I was able to find better solutions later on. This highlights the importance of carefully selecting preprocessing steps to enhance the model's performance without introducing unnecessary complexity. The said techniques are:

- **Removing Elements Instead of Replacing Them.** While removing elements such as mentions, links, and numbers did show an improvement in performance metrics, it became evident that replacing these elements with their generic categories (e.g., replacing mentions or links with "mention" or "link") led to better performance in the model. The reason for this is that removing too much information can lead to the loss of context, which might have been useful for distinguishing between different patterns in the data. The only removal that was consistently beneficial was punctuation, as it doesn't generally contribute to the meaningful features in textual data.

- **Translating slang.** The translation of slang terms consistently resulted in a reduction in performance metrics. Two approaches were attempted: first, using online slang dictionaries to replace slang terms with their corresponding meanings, and second, manually reviewing the training dataset to identify slang words and mapping them to a custom dictionary. Both methods led to a decrease in performance, suggesting that altering slang terms removes valuable contextual in-

formation. This indicates that preserving slang in its original form may be more beneficial for model performance, as it helps the model capture the specific nuances within the data.

- **Spelling Correction.** Spelling correction also led to a decrease in performance metrics. Two methods were attempted: using spelling correction tools such as SymSpell or TextBlob, and manually reviewing the training data to correct misspelled words. Both approaches resulted in a decrease in performance, suggesting that the correction of spelling errors may alter important nuances in the text, ultimately hindering the model's ability to learn effectively from the data.

- **Custom Stop Word List.** As demonstrated in Experiment 2, the removal of stop words led to a decrease in accuracy. In Experiment 2, the default stop word list from TF-IDF was used, which included important words such as "not". To improve the model's performance, I created a custom stop word list, carefully harvested from the training data. However, this approach did not lead to any improvement and ultimately resulted in a decrease in performance, highlighting even more that stop word removal may not always contribute positively to model metrics.

3.1.8. Table of trials. The following table summarizes the different experiments conducted, including variations in text preprocessing, hyperparameters, and resulting performance metrics.

Trial	Text Preprocessing	Hyperparameters	Under/Overfit	Validation score	Test score
E1	None	Default	Overfit	79%	78,937%
E2	Optimal	Default	Overfit	80%	78,947%
E3	Optimal+Stopwords removal+Stemming	Default	Overfit	78%	73,022%
E4	Optimal	Tuned	Underfit	77%	72,508%
E5	Optimal	Tuned	Overfit	78%	80,834%
E6	Optimal	Tuned	Overfit	-	81,47%

Table 1: Summary of the trials conducted and their results

*The "E" in the trials stands for Experiment.

3.2. Hyper-parameter tuning

Hyper-parameter tuning is a crucial step in optimizing the model's performance, as it directly impacts both its ability to learn from data and its generalization to unseen samples. This section outlines the different configurations tested and the final selection of optimal hyper-parameters.

3.2.1. Key Hyper-parameters. During hyperparameter tuning, the primary parameters that were considered are as follows:

- **TF-IDF Vectorizer**
  - **max_df**: Sets the upper frequency limit for terms to be included. Words appearing in more than max _df percent of documents are discarded as too common.

  - **min_df**: Sets the lower frequency threshold. Words that appear in fewer than min_df documents are removed as too rare.
  - **ngram_range**: Determines whether unigrams, bigrams, or larger combinations are used. Larger n-grams can improve phrase understanding but increase feature space dimensionality.
  - **max_features**: Limits the number of features, keeping only the most relevant ones, which helps prevent overfitting.
  - **sublinear_tf**: Applies logarithmic scaling to term frequencies instead of using absolute counts, reducing the impact of highly frequent terms.

- **Logistic Regression**
  - **Penalty**: Specifies the norm of the penalty applied to the model. The default choice is 'l2', which applies L2 regularization (Ridge regression). Setting it to 'l1' applies L1 regularization (Lasso regression), which encourages sparsity in the model. The 'elasticnet' option combines both L1 and L2 penalties, balancing feature selection and weight shrinkage. If set to None, no penalty is applied, meaning the model is trained without regularization.
  - **C**: Inverse of the regularization strength. Higher values reduce regularization and allow more complex models, while lower values increase regularization, simplifying the model.
  - **solver**: The optimization algorithm used for training.
  - **max_iter**: The maximum number of iterations for model convergence. If set too low, the model might not fully converge.

3.2.2. Hyper-parameters tuning results. The results of the tuning process can be categorized into two groups: parameters that had an obvious optimal choice and those that required extensive experimentation.

In the first category, certain parameters had a clear best option based on their intended function and experimental outcomes. The sublinear_tf parameter was an obvious choice, as enabling it reduces the impact of highly frequent terms, leading to better feature scaling. The choice of solver was straightforward as well—through experimentation, all solvers produced similar evaluation metrics, so I opted for SAGA due to its efficiency in handling large datasets. The max_iter parameter was set to 500, as lower values prevented the algorithm from fully converging. Finally, for ngram_range, experimentation determined that using unigrams, bigrams, and trigrams provided the best balance—improving evaluation metrics while keeping the feature space at a manageable size.

In the second category, extensive experimentation was required to find an optimal configuration that avoided both underfitting (which would lead to a drop in evaluation metrics) and overfitting. The first key decision involved setting appropriate thresholds for term frequency to determine which words would be included. Since stopwords were not removed, I tested various combinations following the optimization technique discussed in the next section. The optimal values found were max_df = 0.95 and min_df = 2, meaning that terms appearing in more than 95% of the texts or in fewer than two documents were excluded. Next, I tuned the max_features parameter, ultimately selecting 15,000 features. Higher values led to overfitting, while lower

values resulted in underfitting. Additionally, I optimized the C parameter, which controls the regularization strength. A value of 0.3 was found to provide the best balance, preventing excessive complexity while ensuring the model retained enough flexibility to learn meaningful patterns. Finaly, I applied an L2 penalty, which helps prevent overfitting by discouraging large coefficient values and promoting generalization. L2 regularization adds a penalty proportional to the squared magnitude of the model's weights, effectively reducing the risk of the model memorizing noise in the training data while maintaining its ability to capture relevant patterns.

3.2.3. Prevention of overfitting and underfitting. Through the above-mentioned hyperparameters, the model was able to achieve a good balance between overfitting and underfitting, resulting in stable performance across different datasets. By carefully tuning the key parameters, the model retained enough complexity to capture meaningful patterns without being overly sensitive to noise. This approach allowed for a more robust model capable of generalizing well. Both overfitting and underfitting will be discussed in more detail and analyzed in the upcoming evaluation section.

3.3. Optimization techniques

The optimization techniques used in this project aimed to find the best configuration for the model's hyperparameters, ensuring both efficient performance and generalization. The process involved two key approaches: Grid Search and Manual Tuning.

Initially, I started with a GridSearchCV from scikit-learn over a set of hyper-parameters that were sufficiently diverse to explore a wide range of possible configurations. This method allowed me to identify an approximate range within which the optimal values for each hyperparameter were likely to be found. The main goal of this step was to reduce the search space and narrow down the values to a more manageable range.

Once a reasonable range of hyperparameter values was identified, I proceeded with Manual Tuning. This approach was chosen because it allowed for more flexibility and faster adjustments compared to an exhaustive grid search. With manual tuning, I was able to refine the parameters incrementally, making faster iterations and testing the model's performance on each adjustment. This process helped to find the optimal configuration more effectively.

This two-step process ensured that the hyperparameters were tuned efficiently, balancing between exploration and fine-tuning. By combining these techniques, I was able to optimize the model while preventing both overfitting and underfitting.

3.4. Evaluation

Evaluating the model's performance is a crucial step in understanding its effectiveness in classification tasks. To assess the quality of predictions, multiple evaluation metrics and visualizations are used. These metrics help determine whether the model generalizes well to unseen data, suffers from overfitting, or struggles with class imbalances.
The evaluation process includes:

- **ROC Curve (Receiver Operating Characteristic Curve)**: Measures the model's ability to distinguish between classes across different threshold values.

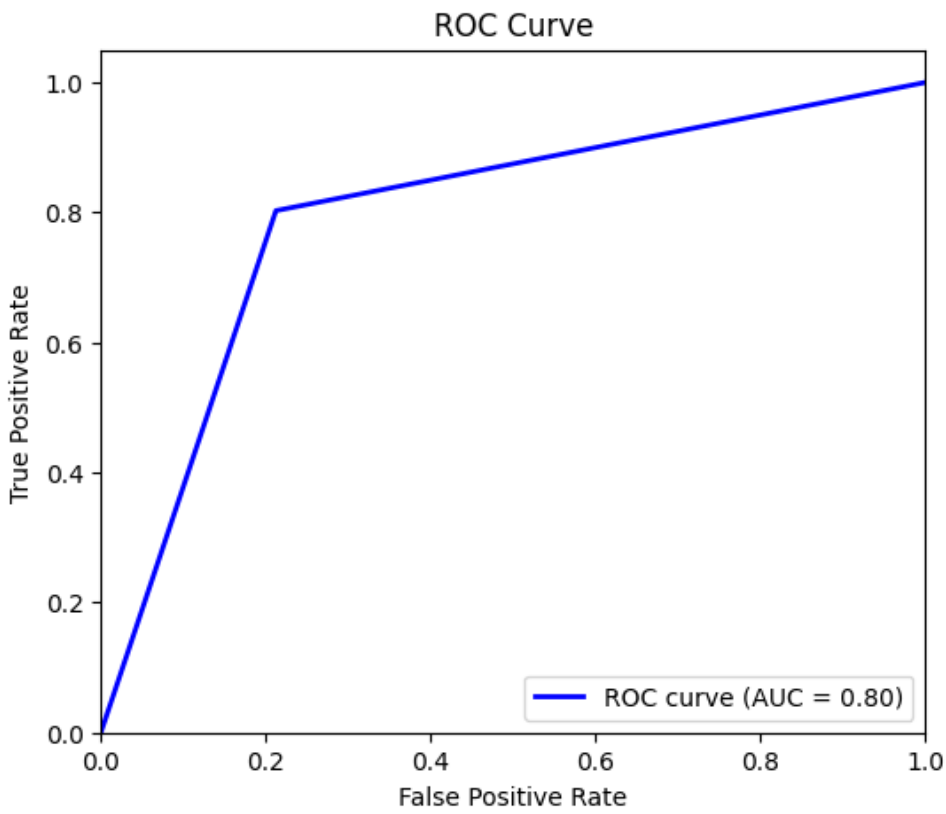
- **Learning Curve**: Demonstrates how the model's performance changes with varying amounts of training data, helping to detect underfitting or overfitting.
- **Classification Report**: Provides a detailed breakdown of key classification metrics such as precision, recall, and F1-score.
- **Confusion Matrix**: Gives insights into the model's misclassifications by showing the distribution of true positives, false positives, true negatives, and false negatives.

Each of these evaluation tools plays a key role in analyzing the strengths and weaknesses of the model. In the following subsections, these metrics and visualizations will be discussed in detail, along with their respective results and interpretations.

3.4.1. ROC curve. The Receiver Operating Characteristic (ROC) curve is a fundamental evaluation tool for assessing the performance of a classification model. It illustrates the trade-off between True Positive Rate (TPR) and False Positive Rate (FPR) across different decision thresholds. A well-performing model will have a curve that approaches the top-left corner, indicating a high true positive rate with a low false positive rate.

To quantify the model's ability to distinguish between classes, the Area Under the Curve (AUC-ROC) is used. A value of 1.0 represents a perfect classifier, whereas 0.5 indicates a model that is no better than random guessing.

The ROC curve for the model is presented. The AUC score achieved was 0.80, indicating moderate to strong discriminative power. The curve demonstrates a steep rise initially, followed by a more gradual increase, suggesting that the model effectively distinguishes between classes at lower false positive rates but struggles slightly as the threshold increases.

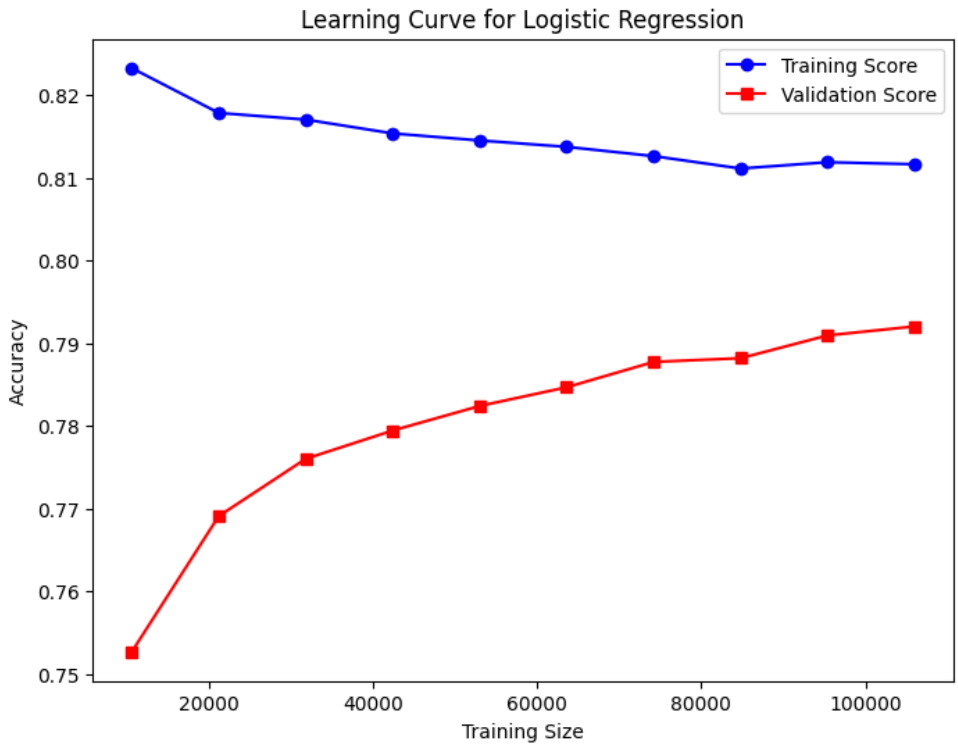


3.4.2. Learning Curve. The learning curve of the model is presented, illustrating the relationship between the training set size and the model's accuracy for both the training set and the validation set. This visualization allows us to assess the model's behavior as the amount of training data increases and to determine whether issues of underfitting or overfitting are present.

From the graph, we observe that the training accuracy starts relatively high but slightly decreases as more data is introduced, eventually stabilizing. At the same time, the validation accuracy steadily increases with more training samples and starts converging towards the training accuracy. This indicates that the model is not underfitting, as the validation accuracy continues to improve with more data, suggesting that the model is learning meaningful patterns rather than struggling to capture the underlying structure of the data.

Furthermore, the gap between the training and validation accuracy is relatively small and does not indicate excessive divergence. In particular, the difference between the two remains approximately 2%, which is within an acceptable range, confirming that the model does not suffer from overfitting. If overfitting were present, we would expect the training accuracy to be significantly higher than the validation accuracy, with the latter stagnating or even decreasing as more training data is added. Instead, the validation accuracy maintains an upward trend and continues to benefit from more training data, confirming that the model generalizes well to unseen examples.

Overall, the learning curve demonstrates that the model achieves a balanced state where it neither oversimplifies the problem nor memorizes the training data, making it a well-generalized and stable classifier.



3.4.3. Clasiffication report. The classification report of the model presents key performance metrics, including precision, recall, and F1-score for both classes. The model achieves an accuracy of 80%, indicating a well-balanced predictive performance across the dataset.

For class 0, the model exhibits a precision of 0.80, meaning that when it predicts class 0, it is correct 80% of the time. Its recall is 0.79, signifying that it successfully identifies 79% of all actual class 0 instances. The F1-score is 0.80, representing the harmonic mean between precision and recall.

For class 1, the precision is 0.79, meaning that 79% of the predicted class 1 instances are correct. The recall for this class is 0.81, meaning the model correctly captures 81% of the actual class 1 instances. The corresponding F1-score is 0.80, indicating a good balance between precision and recall.

The macro average F1-score is 0.80, treating both classes equally, while the weighted average F1-score is also 0.80, accounting for class distribution. These results suggest that the model does not favor one class over the other and maintains consistent performance across both categories.

Overall, the classification report confirms that the model is well-calibrated, with no significant signs of overfitting or underfitting. The precision-recall balance across both classes and the stability of F1-scores indicate that the model generalizes effectively on unseen data.

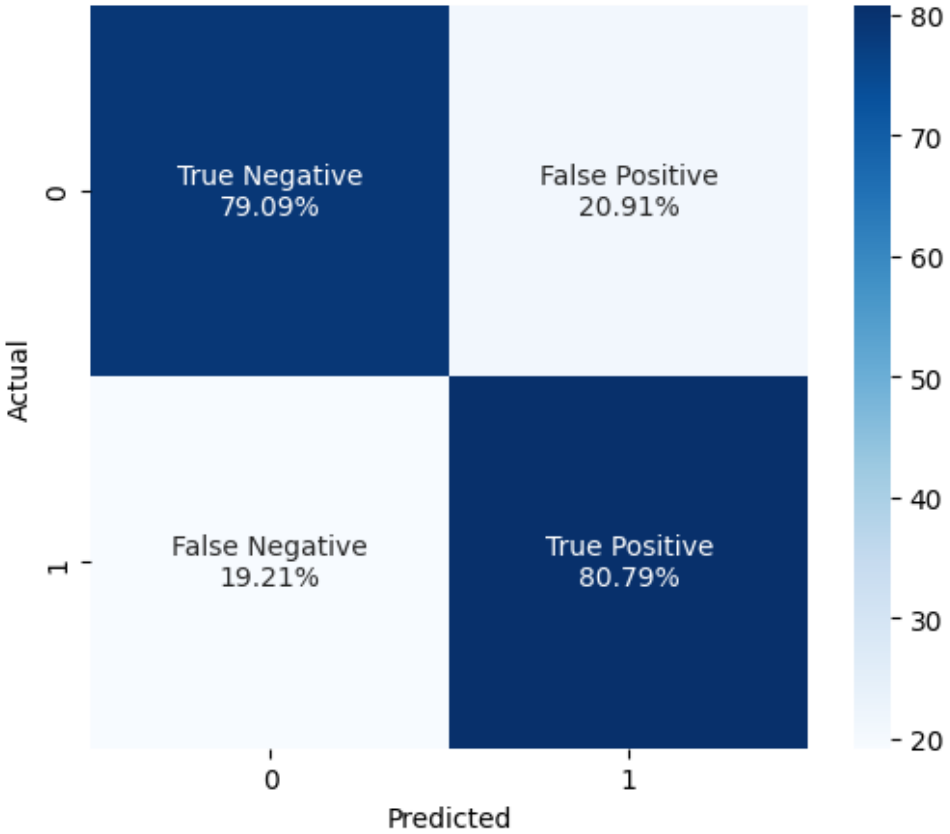
	precision	recall	f1-score	support
0	0.80	0.79	0.80	21197
1	0.79	0.81	0.80	21199
accuracy			0.80	42396
macro avg	0.80	0.80	0.80	42396
weighted avg	0.80	0.80	0.80	42396

3.4.4. Confusion matrix. The confusion matrix provides an in-depth look at the model's classification performance by breaking down its predictions into four categories:

- **True Negatives (TN)**: 79.09% of actual class 0 instances were correctly classified as 0.
- **False Positives (FP)**: 20.91% of actual class 0 instances were misclassified as 1.
- **False Negatives (FN)**: 19.21% of actual class 1 instances were misclassified as 0.
- **True Positives (TP)**: 80.79% of actual class 1 instances were correctly classified as 1.

These results suggest that the model has a balanced performance across both classes, with slightly better recall for class 1 (80.79%) compared to class 0 (79.09%). The false positive rate (20.91%) and false negative rate (19.21%) indicate that there are some misclassifications, but they remain within an acceptable range.

The overall structure of the confusion matrix, in conjunction with the classification report, reinforces that the model is well-trained, does not suffer from overfitting or underfitting, and is capable of effectively distinguishing between the two classes.



4. Results and Overall Analysis

4.1. Results Analysis

Following the systematic development of the model—including thorough text pre-processing, feature engineering, and hyperparameter tuning—the final results provide a clear picture of its performance.

The ROC Curve, with an AUC score of 0.80, indicates a strong ability to differentiate between the two classes. Furthermore, the learning curve analysis confirms that the model is well-trained, showing no signs of underfitting or overfitting, as both the training and validation scores converge smoothly.

The classification report supports these findings, with balanced precision, recall, and F1-score, demonstrating a well-calibrated model with consistent performance across metrics. Additionally, the confusion matrix highlights that the true positive and true negative rates are well-balanced, with minimal misclassification.

Although the results are strong, further experiments could be conducted to explore potential improvements:

- **Text Pre-processing**: Although the text pre-processing steps implemented were robust and contributed to improved performance metrics, there is always potential for further refinement. Given that many of the text replacements were manually extracted from the training data, there is a possibility that certain tokens were overlooked during this process.

- **Alternative Models**: Some other models could achieve better results. While logistic regression performed well, experimenting with more complex models

such as transformer-based architectures (e.g., BERT, RoBERTa) or deep neural networks (e.g., multi-layer perceptrons, recurrent neural networks) could potentially enhance performance. Additionally, ensemble methods might further improve the results.

- **Feature Engineering**: The current approach relies on TF-IDF vectorization, which captures word frequency information. Exploring word embeddings (e.g., Word2Vec, FastText, or contextual embeddings from BERT) could provide a richer representation of text semantics, potentially leading to better performance.

4.1.1. Best trial. The best trial achieved in this project represents a balance between strong performance metrics and the absence of overfitting. Several steps were taken to refine the final model, all of which have been thoroughly explained in the previous sections, accompanied by detailed justifications and evaluation plots.

After implementing these steps, the best model obtained is as follows:

- **Text Pre-processing**: The text preprocessing pipeline, as described in Section 2.1, ensured that the data was cleaned and transformed effectively before feature extraction.
- **TF-IDF Vectorizer Configuration**: The TF-IDF vectorizer was carefully configured to optimize text representation, as explained in Section 3.2. The final parameters used were:

```
TfidfVectorizer(tokenizer=custom_tokenizer, token_pattern=None,
   ngram_range=(1,3), max_df=0.95, min_df=2, max_features=15000,
   sublinear_tf=True)
```

- **Logistic Regression Model**: The selection and tuning of the logistic regression model were thoroughly analyzed in Section 3.2. The final model configuration was:

```
LogisticRegression(C=0.3, solver="saga", random_state=42, max_iter=500)
```

- **Overfitting and Underfitting Prevention**: Strategies to prevent overfitting and underfitting were discussed in Section 3.2.3, while the learning curve analysis in Section 3.4.2 further validated the model's ability to generalize well.
- **Model Evaluation**: The model's performance was assessed in Section 3.4, achieving strong results across multiple evaluation metrics, as presented in Section 3.4.3.

In conclusion, the model attained an accuracy of 80% on the validation dataset and 79,664% on the testing dataset.

5. Bibliography

References

[1] GeeksforGeeks. Understanding logistic regression. *GeeksforGeeks*, 2024. URL.

[2] GeeksforGeeks. Understanding tf-idf (term frequency-inverse document frequency). *GeeksforGeeks*, 2024. URL.

[3] Daniel Jurafsky and James H. Martin. *Speech and Language Processing (3rd Edition) - Chapter 4*. Stanford University, 2021. URL.

[4] Daniel Jurafsky and James H. Martin. *Speech and Language Processing (3rd Edition) - Chapter 5*. Stanford University, 2021. URL.

[5] Scikit learn Developers. *Logistic Regression Documentation*, 2024. URL.

[6] Scikit learn Developers. *TF-IDF Vectorizer Documentation*, 2024. URL.

[7] Grace Mwende Micheni. Nlp (natural language processing) model for twitter sentiment analysis project. *Medium*, 2023. URL.

[8] Giorgos Pantis. Homework 1 lab - artificial intelligence ii. Tutorial, 2025. URL.

[9] Analytics Vidhya. Twitter sentiment analysis: A nlp use case for beginners. *Analytics Vidhya Blog*, 2021. URL.